

PARALLELISING ML – DATA

Adrian Jackson

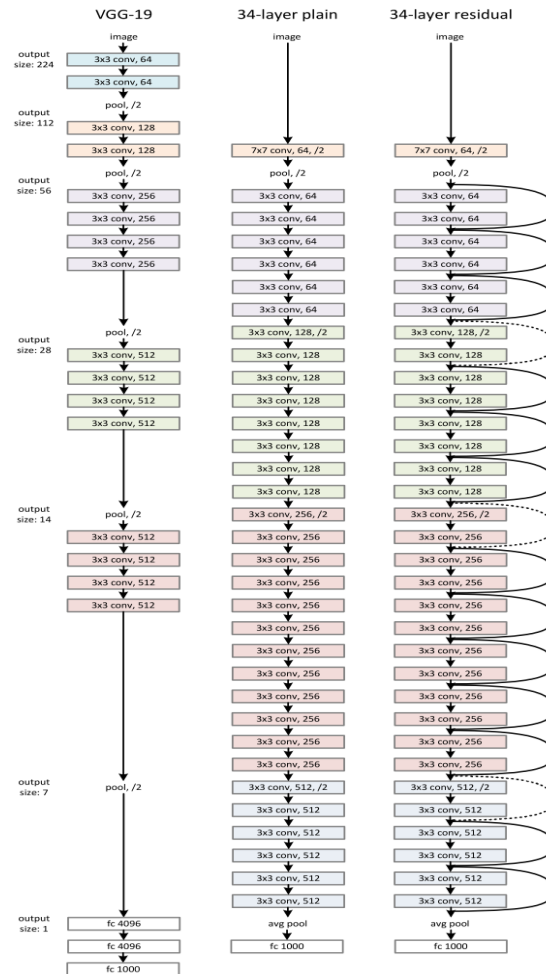
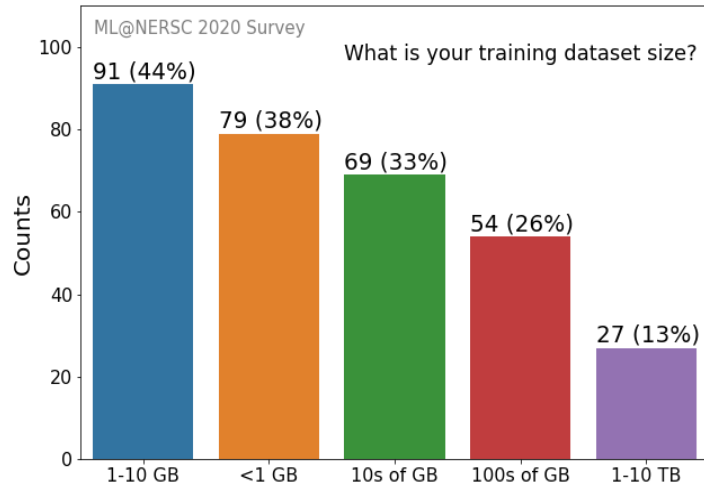
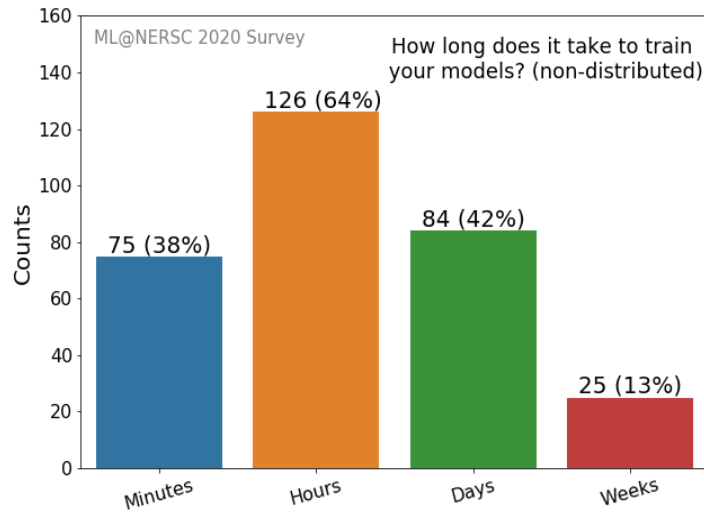
a.jackson@epcc.ed.ac.uk



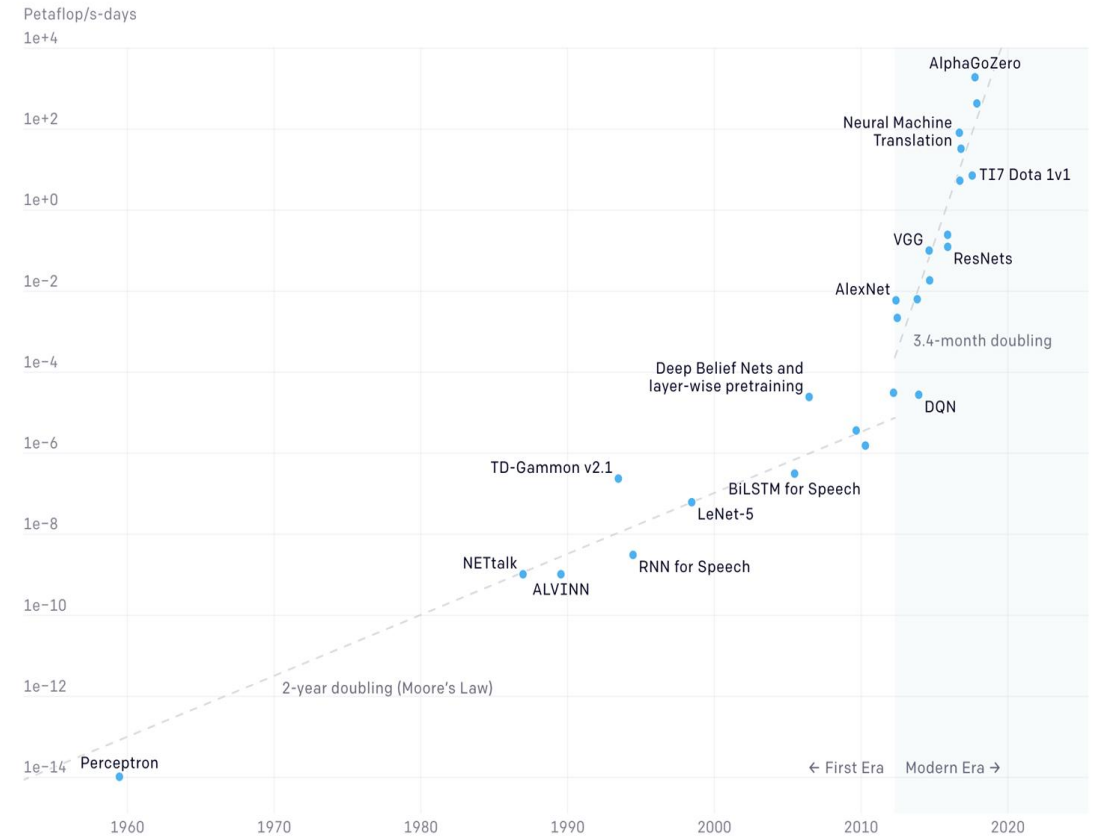
THE UNIVERSITY
of EDINBURGH



Scaling ML

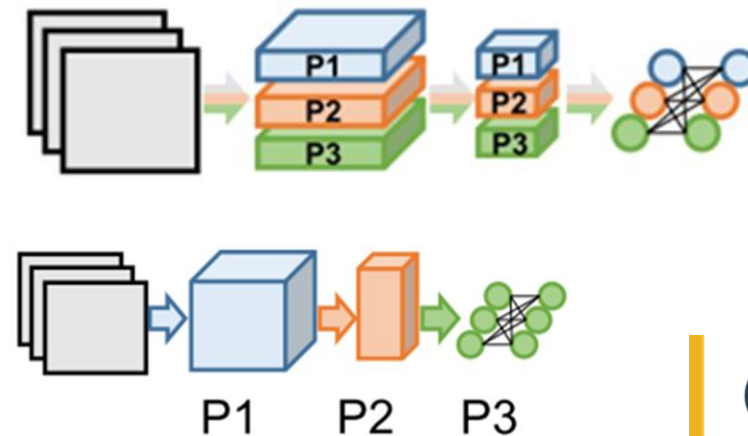
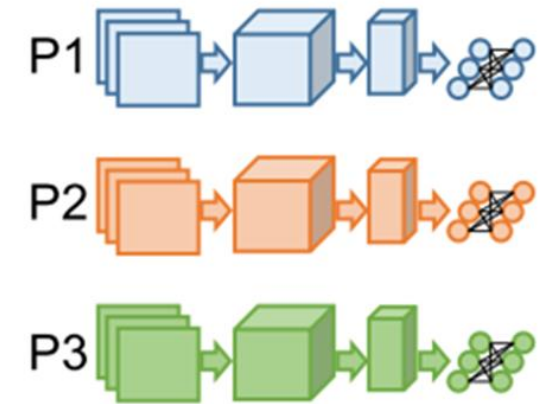


Two Distinct Eras of Compute Usage in Training AI Systems



Multi-resource usage

- Any model or dataset that needs more compute time or memory than a single GPU can provide
 - Need to parallelise some aspect to improve performance
- Standard parallelisation approaches apply
 - Simple parallelisation
 - Inference at scale
 - Advanced task farming
 - Data Parallel: Same model everywhere, different data
 - Distributed data parallelisation
 - Model Parallel: Parts of the model split up
 - Domain decomposition (tensor parallelisation)
 - Distribute function parallelisation
 - Model Parallel: Parts of the model split up
 - Pipelining

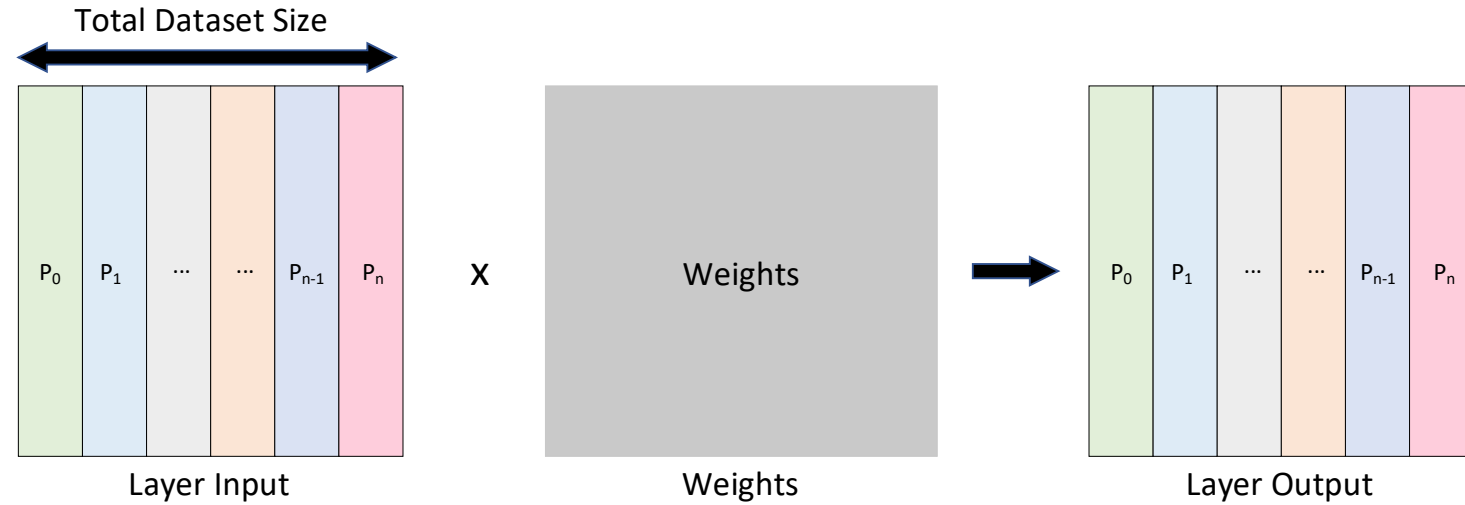


Data parallelism

- Useful to speed up training of large dataset
 - Does not speed up single model
 - Does not help with memory issues
 - Can scale very large datasets
- Data loading, forward pass, and backwards pass can be perfectly scaled
 - With some overheads for data access/networking
- Weights update requires communications
 - Every model instance accumulates its own weights for a given epoch
 - Then need to merge the weights at the end of an epoch to create a single model
 - Conceptually MPI_Allreduce per layer

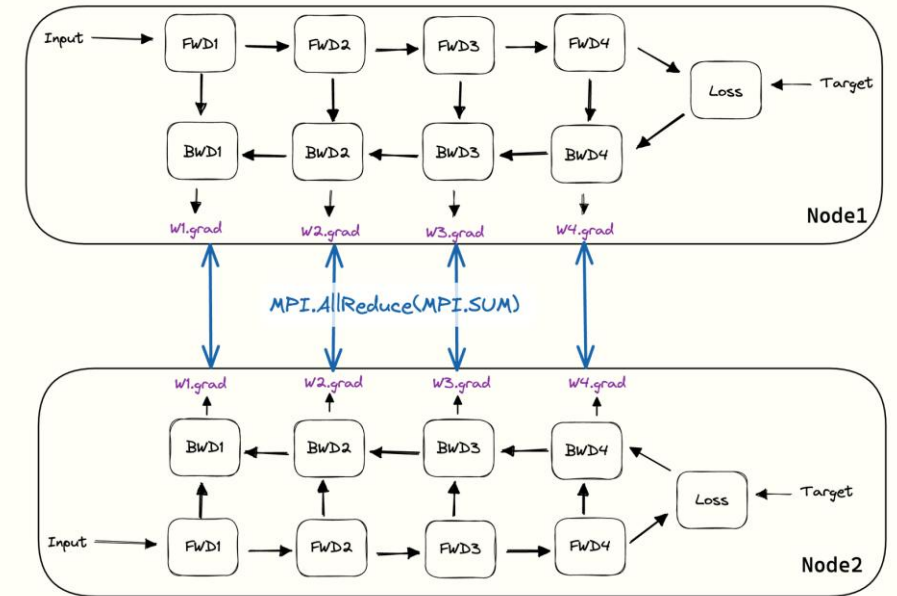
Data parallelism

- General approach is:
 - Independent forward models
 - Coupled backwards pass
 - With varying coupling frequencies
- Works well for basic networks
 - More complicated approaches can have issues
 - If layers depend on total sizes (i.e. RNN), you can get size mismatches on processed data
 - If layers have important state (i.e. a counter or similar) that is not preserved in the replica models (i.e. on the workers)
 - These kinds of caveats depend on the specific frameworks being used



Data parallelism

- Expensive step are the all reduce ops
- Can be some batching of these
 - Inside the framework
 - Improve performance/reduce waiting



<https://siboehm.com/articles/22/data-parallel-training>

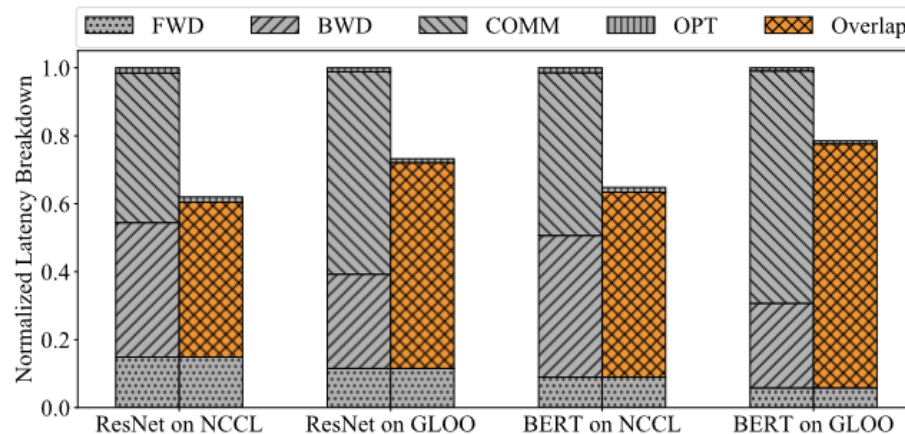
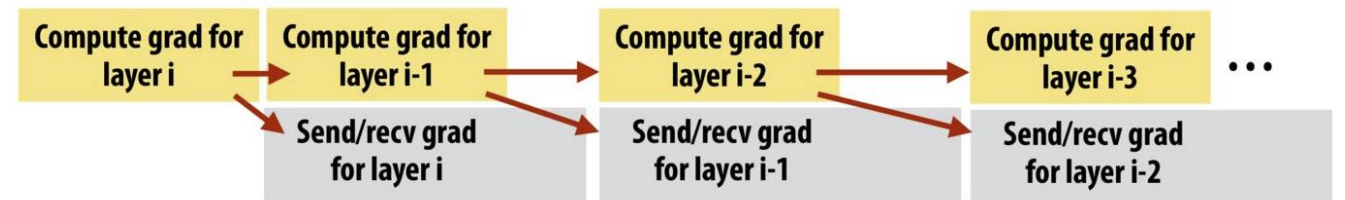


Figure 6: Per Iteration Latency Breakdown

<https://arxiv.org/pdf/2006.15704.pdf>

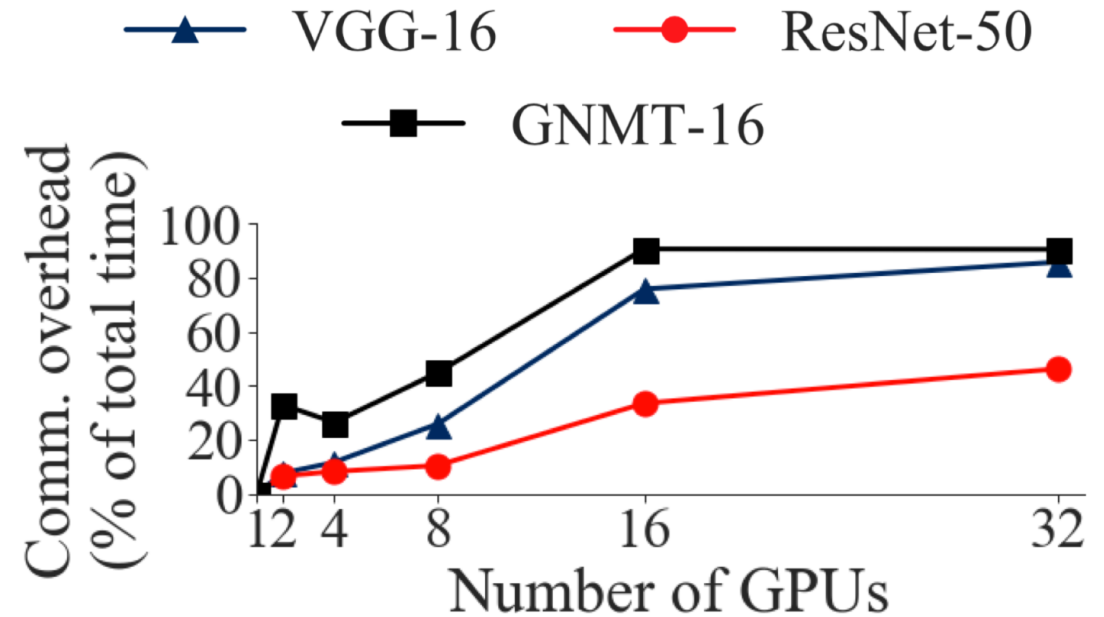


Data parallelism

- Range of possibilities to overlap weights update and data transfers
- Interaction between parallelisation and batch size
 - Weak scaling: Fixing local batch size, overall batches grow with workers (impact generalisability/quality)
 - Strong scaling: Fixed global batch size, local batch sizes get smaller (reduce efficiency)
- Synchronous updates
 - Gradient averaging: Maintains convergence
 - Introduces strong synchronisation
 - Overall batch size grows with parallelism
 - Large-batch training

Scaling

- Can be limited by strict parallelism
 - Comms cost/synch/load balance
 - Similar to load balance issues
 - High performance internal networks
- Future/other functionality
 - Cache coherent memory access
 - High bandwidth networks
- Can reduce the costs with decoupled/desynchronised updates

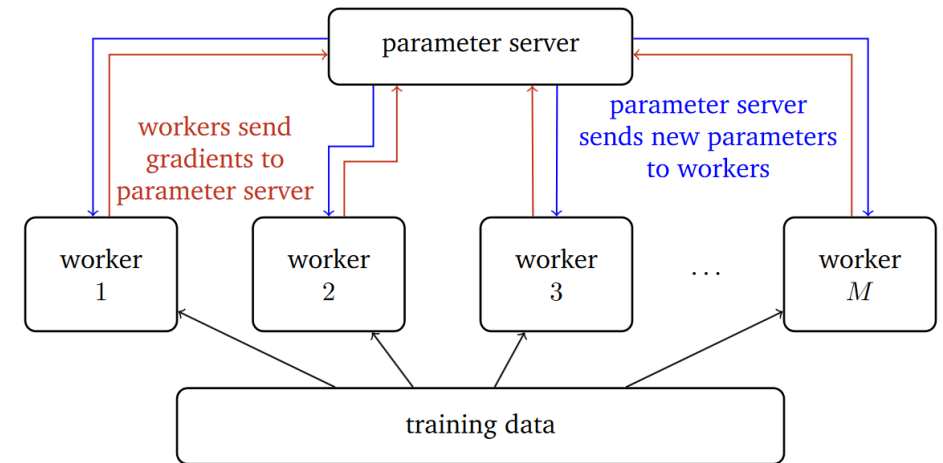


8xV100s with NVLink (AWS)
PyTorch + NCCL 2.4

5

Data parallelism

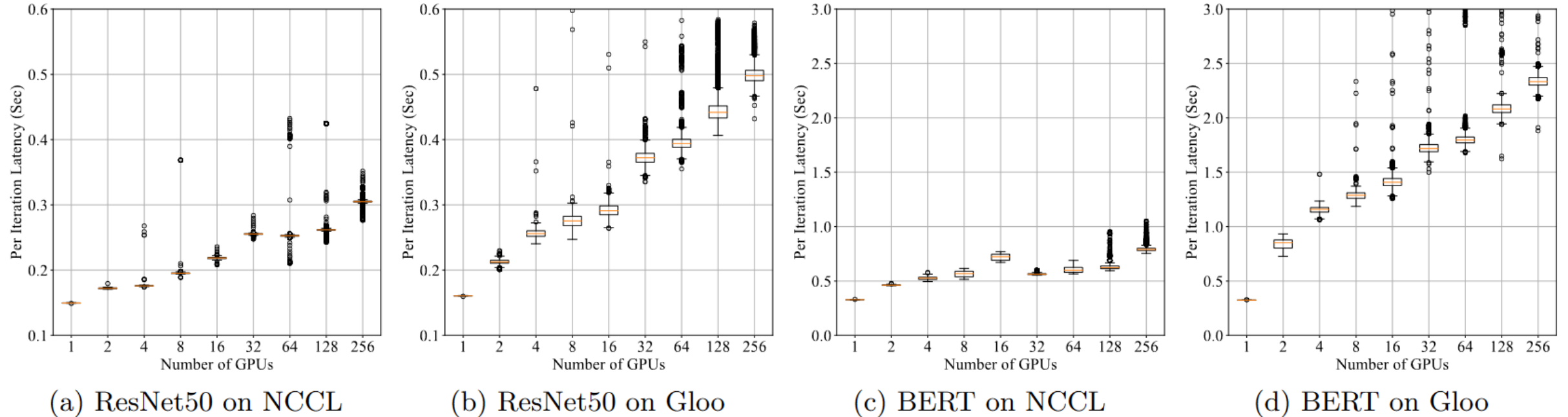
- Asynchronous updates
 - Broadcast style approach: Eager progression
 - Or parameter server approach: Task farming
 - Stale gradients: Can impact convergence
 - Reduces communication requires
 - Decouples synchronisation
- Parameter server approaches
 - RPC: Remote procedure calls
 - Store data on designated servers
 - Model workers fetch/update data as required



Data parallelism

- Delayed gradient updates
 - Send updates infrequently
 - Still update locally
 - Equivalent to increase batch size
- Reduces communication cost
 - Requires tuning batch size and delay factor
- Can benefit from some configuration with the optimiser
 - i.e. momentum settings can smooth issues
 - Some optimisers can adapt to gradient update sizes, i.e. Adam, RMSprop, AdaGrad

Data parallelisation scaling



<https://arxiv.org/pdf/2006.15704.pdf>

Gradient update frequency

- All reduce operations do not have to be done each iteration (i.e. each batch)
 - Can skip updates and accumulate them for less frequent communication
 - Can impact convergence and/or eventual quality
 - Trading speed for convergence

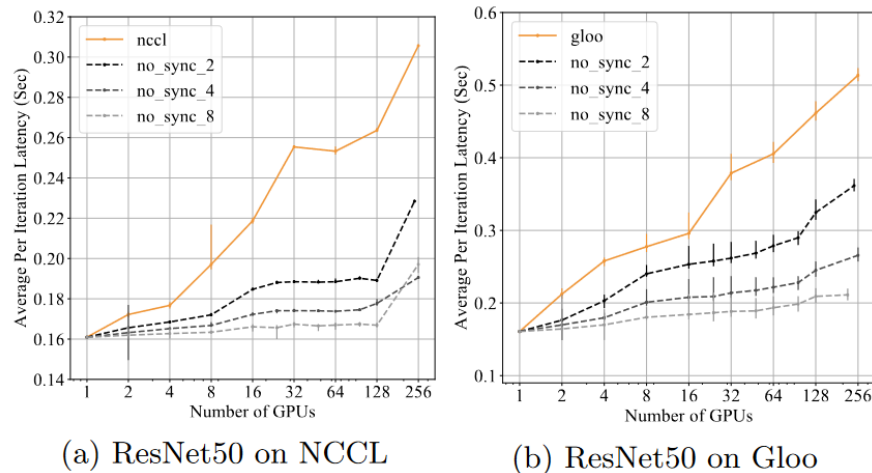


Figure 10: Skip Gradient Synchronization

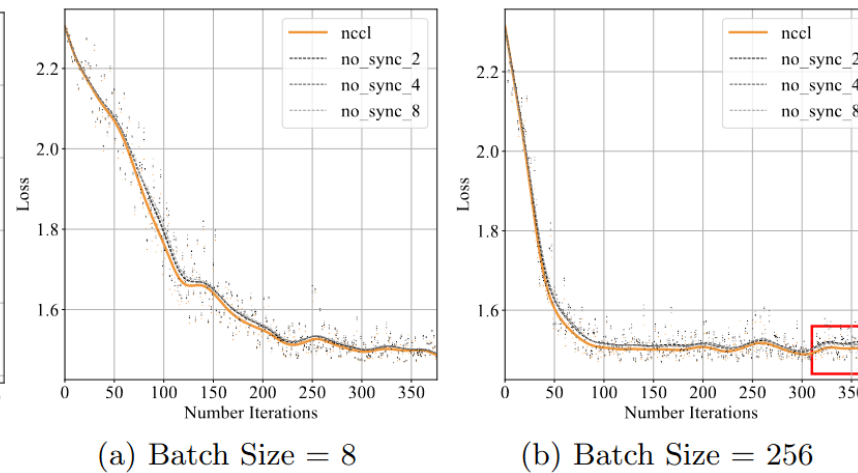


Figure 11: Accuracy with Skipping Synchronization

PyTorch Data Parallelism

- `DataParallel`:
 - multiple GPUs on a single machine
 - Speed up training with minimal code changes
- `DistributedDataParallel`:
 - Potentially fast on single machine, but needs more code changes
 - Can enable multi-node parallelisation
- `torch.distributed.elastic`
 - launch distributed training if errors (like out-of-memory) are expected
 - Resources can join and leave dynamically during training.

PyTorch

- `DataParallel`:
 - Simplest approach, just apply this to the model and PyTorch does the rest

```
model = MyModel()
if torch.cuda.device_count() > 1:
    model = torch.nn.DataParallel(model)
model.to('cuda')
for epoch in range(num_epochs):
    for inputs, targets in data_loader:
        inputs, targets = inputs.to('cuda'), targets.to('cuda')
        # Forward pass
        outputs = model(inputs)
        # Compute loss
        loss = loss_fn(outputs, targets)
        # Backward pass and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

PyTorch DataParallel

- Simple but not necessarily efficient
 - Replicates everything but the input data
 - DataParallel implemented with a single process
 - Multiple Python threads managing the GPUs (GIL issues)
 - Not designed to overlap batching
 - Cannot go beyond a single node
- DistributedDataParallel (DDP)
 - Process parallelisation model, process per GPU
 - Scales beyond a single node
 - Includes a per process optimiser reducing communication required for that step
 - Can overlap batch communications/computation
 - Interfaces with improved communication approaches (NCCL etc...)

PyTorch DistributedDataParallel

- DistributedDataParallel

- Can check if it is available `torch.distributed.is_available()`
- Requires mechanism to communicate between workers
 - Setting up process group
 - `torch.distributed.init_process_group()`

- Two potential approaches

- Direct TCP setup

```
dist.init_process_group(backend, init_method='tcp://10.1.1.20:23456', rank=args.rank, world_size=4)
```

- Or

```
os.environ["MASTER_ADDR"] = "localhost"
```

```
os.environ["MASTER_PORT"] = "12355"
```

```
init_process_group(backend="nccl", rank=rank, world_size=world_size)
```

- Shared filesystem

- Still using TCP, but sharing the details automatically through a file all processes can access
- Required `fcntl` locking on the underlying filesystem

```
dist.init_process_group(backend, init_method='file:///mnt/nfs/sharedfile', world_size=4, rank=args.rank)
```

- Can specify things through environment variables rather than adding to the program

PyTorch DDP

- DDP requires some level of external setup
 - Rank and world size need to be specified for the `init_process_group` call
 - Can be specified by environment variables or manually created (i.e. `MPI4Py`) or defined by a launcher (i.e. `torchrun`)
 - For a single node, can use GPU rank (i.e. process per GPU)
 - Across nodes need global rank
- `torchrun`
 - External script that launches a DDP program
 - Replacement for `python -m torch.distributed.launch my_pytorch_program.py`
 - Specifies the ranks and ensures launching

```
torchrun --nproc_per_node=<NUM_GPUS_PER_NODE> \
        --nnodes=<NUM_NODES> \
        --node_rank=<NODE_RANK> \
        --master_addr=<MASTER_NODE_ADDRESS> \
        --master_port=<MASTER_NODE_PORT> \
        my_pytorch_program.py <SCRIPT_ARGS>
```

PyTorch DDP

- `torchrun` still needs something to start the processes
 - Either run `torchrun` on each machine in the group manually, with identical arguments for the main node
 - Or run it on a system using a batch system (i.e. SLURM), i.e.:

```
#!/bin/bash

#SBATCH --job-name=pytorch_ddp_job      # Job name
#SBATCH --nodes=2                       # Number of nodes
#SBATCH --gpus=4                        # Number of GPUs per node
#SBATCH --ntasks-per-node=4             # Ensure this matches the number of GPUs per node
#SBATCH --time=01:00:00                 # Time limit

# Define MASTER_ADDR and MASTER_PORT
export MASTER_ADDR=$(srun --ntasks=1 hostname | head -n 1)
export MASTER_PORT=45456

# Run the PyTorch script using torchrun
srun torchrun --nnodes=${SLURM_NNODES} --nproc_per_node=${SLURM_NTASKS_PER_NODE} my_pytorch_program.py
```

PyTorch DistributedDataParallel

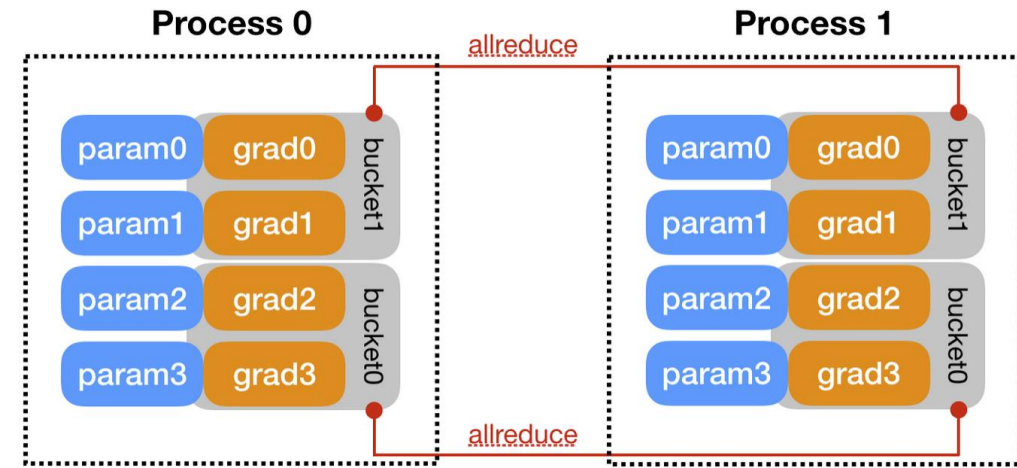
- Can also use MPI for job launching
 - Use mpirun or similar `mpirun -np 4 python3 main.py`
 - Pickup MPI specific environment variables for rank and size, i.e.

```
LOCAL_RANK = int(os.environ['OMPI_COMM_WORLD_LOCAL_RANK'])
```

```
WORLD_SIZE = int(os.environ['OMPI_COMM_WORLD_SIZE'])
```

```
WORLD_RANK = int(os.environ['OMPI_COMM_WORLD_RANK'])
```

- Still need to specify the TCP setting for the main worker
- Timeouts
 - DistributedDataParallel has timeouts to try and deal with process failures
 - Requires workloads to be reasonably load balanced, or will time out
 - Can also increase timeout (parameter to `init_process_group`)
- Communication backends
 - DDP supports different communication backends
 - gloo: Collective communication library, works on both CPU and GPU
 - nccl: NVIDIA Collective Communications Library, optimised for multi-GPU communications, NVLink and Infiniband
 - mpi: Mainly focussed on CPU, depends on the underlying MPI library, a bit more complicated to install



Summary

- Parallelising DNNs to speed them up follows standard parallel approaches
- DataParallel distributes input datasets
 - But the model is replicated
 - And communication is needed to synchronise the weights after updates happen
- Different implementations available
 - Single node/multi-gpu
 - Multi-node/multi-gpu
 - Other approaches
- All aiming for the same overall approach
 - Optimisations around when and how data communicated also undertaken
- More detail on DataParallel and Model parallel in the next lecture